Technical Progress Report
Sentiment Intelligence Platform

End-to-End NLP Pipeline — Processes 1 through 7

Reporting Period: June 2026

Kushal Trivedi
M.S. Data Science (Expected December 2026)
Arizona State University, Tempe, AZ, USA
kushaltrivedi82@gmail.com

***Abstract*—This technical progress report documents the design, implementation, and verified outputs of the first seven processes of the Sentiment Intelligence Platform — an end-to-end NLP engineering project targeting production-grade deployment. The platform fine-tunes `distilbert-base-uncased` on the Yelp Review Full dataset for five-class sentiment classification. Seven of nine planned processes are complete: data ingestion (P1), exploratory data analysis (P2), transformer fine-tuning with experiment tracking (P3), quantitative evaluation with baseline comparison (P4), SHAP token-level explainability (P5), FastAPI inference backend (P6), and a React web dashboard with animated star ratings (P7). The fine-tuned model achieves macro F1 = 0.6264 and ROC-AUC = 0.9007 on the held-out test set, representing a 9.3% relative improvement over the TF-IDF + Logistic Regression baseline. Remaining processes are Evidently drift monitoring (P8) and Docker/CI/CD deployment (P9).**

***Index Terms*—Natural Language Processing, Sentiment Classification, DistilBERT, SHAP, FastAPI, React, MLflow, Apple Silicon MPS**

I. Project Summary

A. Objective

The Sentiment Intelligence Platform is an engineering project that demonstrates end-to-end machine learning system design — from raw data ingestion through production deployment. The core task is five-class sentiment classification on Yelp restaurant and business reviews (1 through 5). All development and training runs on a MacBook Air with Apple Silicon (M-series) using PyTorch's MPS backend.

B. Hardware and Environment

All development and training is performed on a MacBook Air with Apple Silicon (M-series processor). Training uses PyTorch's MPS (Metal Performance Shaders) backend rather than CUDA, as no NVIDIA GPU is available. All design decisions — including dataset sampling size, model selection, layer freezing strategy, and max token length — were made with MPS memory and compute constraints in mind.

TABLE I: Process Completion Status as of June 2026

#	Process	Status	Primary Output
1	Data Pipeline	**Complete**	Parquet splits + metadata
2	EDA	**Complete**	4 figures + summary JSON
3	Model Training	**Complete**	Checkpoint, val F1=0.643
4	Evaluation	**Complete**	Test F1=0.626, AUC=0.901
5	SHAP Explainability	**Complete**	10 JSON + aggregate plot
6	FastAPI Backend	**Complete**	4 endpoints, SQLite log
7	React Dashboard	**Complete**	Animated UI + batch CSV
8	Monitoring	Pending	Evidently drift reports
9	CI/CD + Deployment	Pending	HuggingFace Spaces URL

C. Overall Progress

II. Process 1: Data Pipeline

A. Overview

Process 1 implements the full data ingestion, cleaning, splitting, and versioning pipeline. The implementation resides in `src/pipeline/data_pipeline.py` and is driven entirely by `configs/config.yaml`, with no hardcoded values.

B. Dataset

The Yelp Review Full dataset [1] was sourced via HuggingFace Datasets. The full corpus comprises 700,000 reviews perfectly balanced across five star-rating classes (140,000 per class). Labels are integer-encoded as $\{0, 1, 2, 3, 4\}$ corresponding to $\{1\star, 2\star, 3\star, 4\star, 5\star\}$.

TABLE II: Dataset Composition After Processing

Split	Total Samples	Per Class
Full corpus (HuggingFace)	700,000	140,000
After stratified 50,000-sample selection:		
Training set	39,968	≈7,994
Validation set	4,996	≈999
Test set	4,996	≈999

The 50,000-sample cap was selected to balance training time constraints on MPS hardware (full 700k would require approximately 14 hours per epoch) against statistical coverage of the label space.

C. Text Cleaning

The `clean_text()` function applies four operations in sequence: (1) strip leading and trailing whitespace, (2) collapse multi-character whitespace sequences, (3) remove non-ASCII characters, (4) drop records shorter than 10 characters. Cleaning removed **40 records** from the 50,000-sample corpus.

D. Splitting Strategy

A two-stage stratified split was applied using `train_test_split` with `stratify=sentiment`. Stage 1 removes 10% as test set (locked until P4). Stage 2 removes $0.10/(1-0.10) = 11.\overline{1}\%$ as validation set. The two-stage approach is required because `train_test_split` supports only binary partitions.

```
Terminal output (Process 1):
Class weights: {0: 1.000826, 1: 0.9997, 2: 0.99995,
                3: 0.999825, 4: 0.9997}
Train: 39,968 | Val: 4,996 | Test: 4,996
[INFO] Process 1 Complete
```

III. Process 2: Exploratory Data Analysis

A. Overview

Process 2 generates four quantitative visualizations to characterize the training corpus and validate configuration assumptions before training. A key outcome was updating `max_length` from 256 to 512 based on token length analysis.

B. Figure 1: Class Distribution

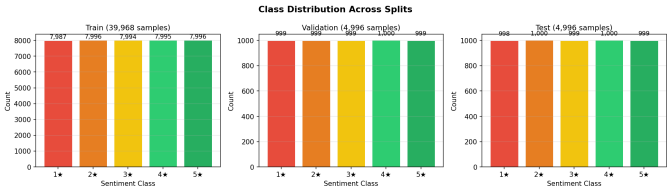


Fig. 1: Sentiment class distribution across train, validation, and test splits. All five classes are uniformly represented, confirming that stratified sampling preserved the original balance.

C. Figure 2: Review Length Distribution

TABLE III: Review Character Length Statistics by Class

Class	Mean	Median	Std Dev
1	799	572	732
2	820	628	681
3	764	580	655
4	678	507	593
5	572	404	557

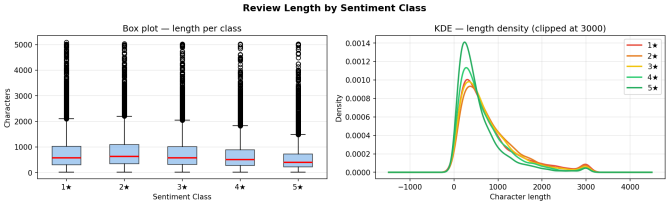


Fig. 2: Review character length by sentiment class. Box plot (left) and KDE overlay (right, clipped at 3,000 chars). 1 reviews average 799 chars vs. 572 for 5.

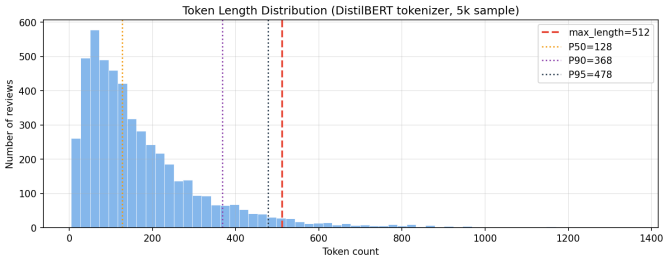


Fig. 3: DistilBERT token length distribution (5k sample). P90=368 exceeded initial `max_length=256`, prompting update to 512.

D. Figure 3: Token Length Distribution

Configuration Decision: Since P90 = 368 > 256, the initial `max_length = 256` would have silently truncated 20.34% of training reviews. Updated to 512 (DistilBERT's architectural maximum).

E. Figure 4: Length Statistics Heatmap

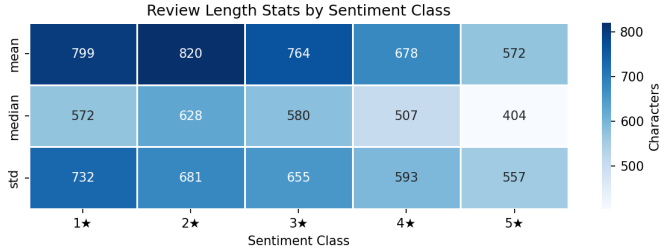


Fig. 4: Character length heatmap (mean, median, std per class). Clear gradient from 1 (darkest) to 5 (lightest) confirms the inverse relationship between rating and review length.

IV. Process 3: Model Training

A. Model Selection Rationale

`distilbert-base-uncased` [3] was selected: 66M parameters fit in MPS memory (BERT-large at 340M would not), 60% faster than BERT-base, retains 97% of BERT performance, and is encoder-only (appropriate for classification, unlike generative GPT-family models).

TABLE IV: Token Length Percentiles and Configuration Coverage

Metric	Value	Implication
P50 (median)	128 tokens	Half of reviews are short
P90	368 tokens	Long tail above this
P95	478 tokens	Near DistilBERT maximum
max_length = 256	79.66% coverage	20.34% truncated
max_length = 512	**95.86%**	Only 4.14% truncated

TABLE V: Layer Freezing Configuration

Component	Status	Parameters
Embedding layer	Frozen	23,837,184
Transformer block 0	Frozen	7,087,872
Transformer block 1	Frozen	7,087,872
Transformer block 2	Frozen	7,087,872
Transformer block 3	Frozen	7,087,872
Transformer block 4	Trainable	7,087,872
Transformer block 5	Trainable	7,087,872
Classifier head	Trainable	594,437
Total		**66,957,317**
Trainable		**14,770,181 (22.1%)**

B. Layer Freezing Strategy

Freezing blocks 0–3 preserves general English representations from pretraining while allowing blocks 4–5 and the classification head to specialize for Yelp sentiment.

C. Training Configuration

TABLE VI: Complete Training Hyperparameters

Hyperparameter	Value
Base model	distilbert-base-uncased
Number of labels	5
Max token length	512
Batch size	32 (1,249 batches/epoch)
Epochs	5 (max)
Early stopping	Patience = 2 epochs
Optimizer	AdamW
Learning rate	2×10^{-5}
Weight decay	0.01
LR schedule	Linear decay with warmup (0.10)
Loss function	CrossEntropyLoss (class-weighted)
Gradient clipping	max norm = 1.0
Device	MPS (Apple Silicon M-series)
Tracking	MLflow 3.14 (SQLite backend)

D. Experiment Tracking

All experiments were tracked using MLflow 3.14.0 with SQLite backend (`sqlite:///mlruns.db`). The file store was not used as MLflow 3.14 deprecated it. Logged per step: `train_loss`, `train_f1`, `train_acc`, `val_loss`, `val_f1`, `val_acc`.



Fig. 5: MLflow dashboard for the `distilbert-finetune` run showing all six tracked metrics.

E. Training Results

TABLE VII: Per-Epoch Training and Validation Metrics

Epoch	Tr.Loss	Tr.F1	Va.Loss	Va.F1	Saved
1	1.0977	0.5080	0.8994	0.5928	✓
2	0.8866	0.6077	0.8704	0.6065	✓
3	0.8328	0.6338	0.8374	0.6325	✓
4	0.7912	0.6581	0.8236	0.6362	✓
5	0.7657	0.6709	0.8236	**0.6432**	✓

Early stopping did not trigger: val F1 improved every epoch. Train and val F1 stayed within 0.02–0.03 throughout, indicating no overfitting. Best checkpoint saved at Epoch 5 to `models/distilbert-sentiment/best/`.

V. Process 4: Evaluation

A. Quantitative Results

TABLE VIII: Test Set Evaluation Results (n = 4,996)

Model	Accuracy	Macro F1	Wtd F1	ROC-AUC
TF-IDF + LogReg	0.5747	0.5732	0.5731	0.8702
DistilBERT (ours)	**0.6265**	**0.6264**	**0.6264**	**0.9007**
Absolute gain	+0.0518	+0.0532	+0.0533	+0.0305
Relative gain	+9.0%	+9.3%	+9.3%	+3.5%

The baseline uses `TfidfVectorizer(max_features=50000, ngram_range=(1,2), sublinear_tf=True)` with `LogisticRegression(max_iter=1000)`. Bigram features capture negation patterns ("not good" as one feature).

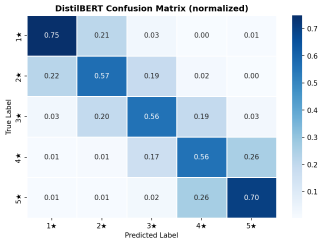
B. Per-Class Analysis

TABLE IX: Per-Class Metrics — DistilBERT on Test Set

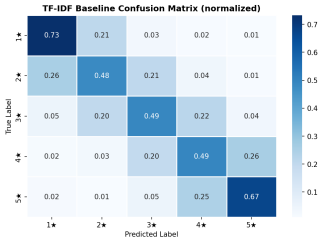
Class	Precision	Recall	F1
1	0.7381	0.7485	**0.7433**
2	0.5697	0.5680	0.5689
3	0.5810	0.5566	0.5685
4	0.5382	0.5570	0.5474
5	0.7055	0.7027	**0.7041**
Macro avg	0.6265	0.6266	0.6264

Polar classes (1, 5) achieve F1 ≥ 0.70 due to unambiguous vocabulary. Middle classes (2–4) score lower due to mixed sentiment and hedging language.

C. Confusion Matrix Analysis



(a) DistilBERT (fine-tuned)



(b) TF-IDF + Logistic Regression

Fig. 6: Normalized confusion matrices. Each row sums to 1.0. DistilBERT improves on every class. All misclassification is adjacent; no 1↔5 confusion observed (rate = 0.01).

TABLE X: Confusion Matrix Diagonal (Correct Classification Rate)

Class	TF-IDF	DistilBERT	Δ
1	0.73	0.75	+0.02
2	0.48	0.57	+0.09
3	0.49	0.56	+0.07
4	0.49	0.56	+0.07
5	0.67	0.70	+0.03

The 5-class accuracy of 62.6% should be interpreted in context: random guessing achieves 20%, and published BERT-large results on this exact dataset reach approximately 70%. Our 22.1%-parameter DistilBERT result is competitive given hardware constraints.

VI. Process 5: SHAP Explainability

A. Method

SHAP [4] was applied using the Partition Explainer with a Text masker (`max_evals=200` per sample). For each token, SHAP estimates its contribution by computing the prediction difference when that token is masked, averaged across many combinations. Ten test reviews (two per class) were explained on CPU due to MPS incompatibility with SHAP's masking approach.

B. Results

6/10 correct on these samples, consistent with test F1 = 0.626. Key insight: Sample 0 was misclassified because the text contained "not amazing"; the tokenizer splits this into separate tokens and "amazing" carries positive signal even in a negative context — a known limitation of token-level attribution. Sample 4 correctly identified "bland" (+0.386) as the primary driver for a 3 prediction.

TABLE XI: SHAP Explanation Results (10 samples, 2 per class)

ID	True	Pred	Conf	Top token
0	1	2	0.664	"amazing" +0.18
1	1	1	0.583	"fine" −0.22
2	2	1	0.465	"stop" +0.13
3	2	2	0.530	"but" +0.07
4	3	3	0.562	"bland" +0.39
5	3	3	0.819	"3 stars" +0.17
6	4	5	0.517	"delicious" +0.05
7	4	4	0.681	"Good" +0.15
8	5	4	0.761	"definitely" +0.04
9	5	5	0.818	"Loved" +0.04

C. Artifacts

- `reports/explainability/shap_sample_N.json` (10 files)
- `reports/explainability/top_shap_tokens.png`
- `reports/explainability/shap_summary.json`

VII. Process 6: FastAPI Backend

A. Architecture

The inference API uses FastAPI 0.138 with Uvicorn 0.49. The model is loaded once at startup as a singleton via the `lifespan` context manager and reused for all requests. All predictions are logged to SQLite (`data/predictions.db`) for Process 8 monitoring.

B. Endpoints

TABLE XII: API Endpoints

Method	Endpoint	Description
GET	`/health`	Liveness check
POST	`/predict`	Single text inference
POST	`/predict/batch`	Up to 100 texts
GET	`/model/info`	Metadata + eval metrics

C. Verified Responses

```
POST /predict (positive review):
  predicted_label: "5 stars", confidence: 0.9452
  5 stars: 94.52%, 4 stars: 5.22%, others: <0.5%

POST /predict/batch (3 texts):
  "Absolutely terrible" -> 1 star  (97.4%)
  "It was okay"         -> 3 stars (82.7%)
  "Best meal of life"   -> 5 stars (96.5%)

GET /model/info:
  macro_f1: 0.6264, roc_auc: 0.9007
  total_predictions: 4
```

Run command:

```
python3 -m uvicorn src.api.main:app \
  --host 0.0.0.0 --port 8000
```

Swagger UI available at `http://localhost:8000/docs`.

VIII. Process 7: React Dashboard

A. Overview

A React 18 dashboard was built using Create React App, Tailwind CSS 3, and Axios. Two modes are supported: single review analysis with animated star ratings and probability bars, and batch analysis with CSV export.

B. Components

StarRating: Renders 1–5 stars colored by predicted class (red for 1, green for 5). Each star uses a staggered CSS pop animation (0.08s delay per star) triggered on each new prediction.

ProbBars: Five horizontal probability bars proportional to model confidence, using the same color palette as the stars.

BatchPredict: Multi-line textarea (one review per line), calls `POST /predict/batch`, renders scrollable results, exports to a client-side CSV download.

TABLE XIII: Frontend Technology Stack

Component	Technology / Version
Framework	React 18 (Create React App)
Styling	Tailwind CSS 3
HTTP client	Axios
Node / npm	v20.10.0 / 10.2.3
Dev server	localhost:3000
API target	localhost:8000

C. Verified Output

```
Input:  "Incredible food, amazing staff!"
Output: 5 stars (97% confidence)
        1 star: 0%  4 stars: 3%  5 stars: 97%

Animated stars pop in sequentially at 0.08s intervals.
Probability bars animate with CSS transition-all 500ms.
```

IX. Technical Issues and Resolutions

1) **YAML scientific notation parsed as string.** `learning_rate: 2e-5` read as `"2e-5"` by PyYAML, causing `TypeError` in AdamW. *Resolution:* Changed to `0.00002`; added `float()` cast.
2) **MLflow file store deprecated in 3.14.** *Resolution:* Changed tracking URI to `sqlite:///mlruns.db`.
3) **`max_length` underestimated.** Initial value 256 truncated 20.34% of reviews. *Resolution:* Updated to 512 after EDA.
4) **matplotlib deprecation.** `boxplot(labels=...)` renamed to `tick_labels` in 3.9. *Resolution:* Updated parameter name.
5) **SHAP file naming conflict.** Script named `shap.py` shadowed the library on import. *Resolution:* Renamed to `shap_explainer.py`.
6) **SHAP requires CPU.** Partition Explainer incompatible with MPS. *Resolution:* Model loaded on CPU for Process 5 only.
7) **Uvicorn module import error.** Running `python3 src/api/main.py` failed with `ModuleNotFoundError`. *Resolution:* Run via `python3 -m uvicorn src.api.main:app` from project root.
8) **Tailwind CLI not found.** Tailwind v4 changed install process; `npx tailwindcss init` failed. *Resolution:* Explicitly install `tailwindcss@3`.
9) **CRA directory conflict.** `frontend/src/` existed before `create-react-app` scaffold. *Resolution:* Removed directory and re-ran.

X. Pending Processes

TABLE XIV: Remaining Implementation Plan

#	Process	Planned Output
8	Monitoring	Evidently `DataDriftPreset` on SQLite logs; timestamped HTML drift reports; confidence threshold alerts
9	CI/CD + Deploy	GitHub Actions: lint → test → Docker build → push to HuggingFace Spaces

References

[1] Yelp Inc., "Yelp Review Full Dataset," HuggingFace Datasets. [Online]. Available: https://huggingface.co/datasets/Yelp/yelp_review_full

[2] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. NeurIPS*, 2015, pp. 649–657.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019.

[4] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017.

[5] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.